

Autoware 规划控制总览

从目标点到方向盘控制命令

Goal -> Route -> Path -> Trajectory -> Control

这一集解决什么问题

给自动驾驶车一个目标点：

它最后是怎样一步步变成
方向盘、油门、刹车控制命令的？

本集先建立全局框架，不急着读源码，也不急着推复杂公式。

为什么先讲总览

Autoware 规划控制层里有很多模块：

- Mission Planner
- Behavior Planner
- Motion Planner
- Velocity Smoother
- Controller
- Vehicle Interface

如果没有主线，源码越看越多，反而越看越乱。

一个最简单的任务

车辆当前在 A 点
目标位姿在 B 点

问题：

车辆应该怎么开过去？

目标点不能直接控制车辆

目标点没有告诉车辆：

1. 应该经过哪些车道
2. 在车道内具体走哪条线
3. 每一段应该开多快
4. 当前方向盘应该转多少
5. 当前应该加速还是减速

规划控制总链路

Goal

- > Route
- > Path
- > Trajectory
- > Control Command
- > Vehicle

这张图是整套课程的主线。

本质是逐层翻译

我要去哪里

- > 应该经过哪些车道
- > 在车道里走哪条线
- > 每个位置以多快速度通过
- > 当前车辆怎么执行

抽象目标越往后，越接近底盘可执行命令。

四个核心概念

名称	回答的问题	典型结果
Route	经过哪些车道	lanelet 序列、route sections
Path	在车道里走哪条线	几何路径点
Trajectory	每个点以多快速度通过	带速度轨迹点
Control	当前怎么操作车辆	转向、加速度、制动

Route：车道级路线

Route 回答：

从当前位置到目标点，应该经过哪些车道？

输入通常包括：

- 当前车辆位姿
- 目标位姿
- Lanelet 地图
- 途经点或任务约束

Path：车道内的几何走线

Path 回答：

在 Route 给定的道路空间里，
车辆具体沿哪条几何线行驶？

它需要考虑：

- 车道边界
- 障碍物
- 路径平滑性
- 车辆能否跟踪

Trajectory: 带速度的路径

Trajectory 可以理解为:

Path + Velocity

轨迹点常包含:

x, y, yaw, velocity, acceleration

同一条几何路径，不同速度曲线会产生完全不同的驾驶行为。

Control Command: 当前执行命令

Control Command 回答:

车辆此刻应该怎么执行?

常见输出:

- 转向角或转向角速度
- 目标加速度
- 制动命令
- 档位命令

模块地图

Goal

- > Mission Planner
- > Behavior / Start Planner
- > Motion Planning / Path Optimization
- > Velocity Smoother
- > Controller
- > Vehicle Interface

每个模块都只解决链路上的一部分问题。

Mission Planner 做什么

目标点 + 当前位姿 + Lanelet 地图
->
车道级 Route

核心步骤：

1. 找起点所在车道
2. 找目标所在车道
3. 在车道图上搜索
4. 输出 route sections

Lanelet 地图为什么重要

Lanelet 地图提供：

- 车道边界
- 中心线
- 行驶方向
- 车道连接关系
- 可换道关系
- 交通规则属性

没有拓扑关系，系统就不知道哪些车道能连起来。

Behavior / Start Planner 做什么

它回答更局部的行为问题：

- 是否需要换道
- 是否需要停车
- 是否需要靠边
- 是否需要从路边起步
- 是否需要从自由空间回到车道

Mission Planner 给方向，行为规划决定具体场景动作。

Hybrid A* 解决什么

普通图搜索常把状态看成：

(x, y)

车辆局部规划常需要：

$(x, y, \text{yaw}, \text{gear})$

因为车辆有朝向、最小转弯半径，还可能需要前进和倒车。

路径为什么需要平滑

上游路径可能来自：

- 车道中心线
- 路径拼接
- 避障绕行
- 离散采样
- 历史轨迹复用

轻微折角会放大成曲率抖动和转向抖动。

Elastic Band 的位置

原始离散路径

->

横向微调路径点

->

更光顺的路径

直觉：

让路径像一根橡皮筋一样变平滑，
但不能随意偏离原路径。

MPT 轨迹优化的位置

MPT 更关注：

- 局部参考线
- 车辆模型
- 道路边界
- 软约束
- 轨迹连续性
- warm start

它让轨迹更适合后续控制器跟踪。

Velocity Smoother 做什么

几何轨迹

- + 速度上限
- + 当前车速/加速度
- + 运动约束

->

满足速度、加速度、jerk 的轨迹

路径回答“走哪里”，速度平滑回答“怎么快慢变化”。

速度不能硬裁剪

错误直觉：

```
v = min(v, speed_limit)
```

真实车辆还要满足：

$$\begin{aligned}0 &\leq v \leq v_{\max} \\ a_{\min} &\leq a \leq a_{\max} \\ j_{\min} &\leq j \leq j_{\max}\end{aligned}$$

Controller 做什么

当前车辆状态 + 目标轨迹
->
当前控制命令

横向控制关注：

- 横向误差
- 航向误差
- 曲率或转角

纵向控制关注：

- 速度误差
- 加速度
- 制动

Pure Pursuit: 几何控制

核心理念：

不要追最近点，
而是追轨迹前方的前瞻点。

在车辆坐标系下：

$$\kappa = \frac{2y}{x^2 + y^2}$$

再由自行车模型转换成转角。

MPC：优化控制

MPC 的核心想法：

预测未来一段车辆运动，
选择一串控制输入，
让未来误差尽量小，
同时满足转角和变化率约束。

典型结果：

$$U^* = [u_0, u_1, \dots, u_{N-1}]$$

实际只执行第一个控制量。

车辆模型为什么重要

车辆不是点，也不能横着平移。

常用自行车模型：

$$\dot{x} = v \cos \psi$$

$$\dot{y} = v \sin \psi$$

$$\dot{\psi} = \frac{v}{L} \tan \delta$$

Hybrid A*、Pure Pursuit、MPC 都会用到这个直觉。

为什么优化问题反复出现

很多模块都在做：

最小化某个代价
同时满足一些约束

例子：

- 路径尽量平滑，但不能偏离太多
- 速度尽量高，但 jerk 不能太大
- 控制误差尽量小，但转角不能超限
- 轨迹尽量贴近参考，但不能越界

调试时先问链路问题

如果车的行为不对，先定位是哪一层：

现象	优先检查
路线不对	地图、目标点、Mission Planner
局部路径绕得奇怪	行为规划、局部规划、障碍物
路径有折角	平滑器、重采样、拼接
刹车突兀	Velocity Smoother、纵向控制
转向抖动	轨迹曲率、横向控制、车辆状态

正确学习顺序

建议按这条路学：

1. 总览
2. Lanelet 与 Mission Planner
3. Hybrid A* 与 Pull Out
4. Elastic Band 路径平滑
5. MPT 轨迹优化
6. Velocity Smoother
7. Pure Pursuit
8. 横向 MPC
9. 总复盘

后续课程路线

精简版 8 集：

集数	主题
1	规划控制总览
2	Lanelet 与 Mission Planner
3	Hybrid A* Pull Out
4	Elastic Band 路径平滑
5	Velocity Smoother
6	Pure Pursuit
7	横向 MPC
8	MPT 与总复盘

本集总结图

Goal

- > Route
经过哪些车道
- > Path
在车道里走哪条线
- > Trajectory
每一段开多快
- > Control Command
当前怎么执行

三句话总结

第一，规划控制是一条链，不是一个单独算法。

第二，Route、Path、Trajectory、Control 处在不同抽象层。

第三，学习源码前，先搞清楚每个模块的输入、输出和物理意义。

课后思考

1. Route 和 Path 最大区别是什么?
2. 为什么速度规划不能只做 `min(v, speed_limit)` ?
3. 为什么控制器不应该重新决定路线?
4. 如果车辆转向抖动, 应该只看控制器吗?
5. 为什么车辆模型会影响局部路径规划?

下一集预告

Lanelet 与 Mission Planner

下一集会讲：

1. 道路如何变成 Lanelet 图
2. 起点和目标点如何匹配到车道
3. 最短路线如何生成
4. 为什么输出是 route sections

下一次给 Codex 的指令

生成下一集时，直接输入：

请根据 AUTOWARE规划控制层视频教程制作指南.md、AUTOWARE规划控制层第1集_规划控制总览PPT.md、concept/mission_planner_route_planning_blog.md 和 concept/lanelet.md，生成第 2 集“Lanelet 与 Mission Planner”的 PPT 制作稿。

要求：

1. 在项目根目录生成 Markdown 格式 PPT 文件；
2. 使用 Marp 风格，以 --- 分隔每一页；
3. 每页包含页面展示内容、讲解备注、配图建议；
4. 面向新手，原理为主，代码为辅；
5. 生成完成后，在文件末尾给出生成第 3 集的指令。